

Técnicas Digitales 2 - Trabajo práctico Nro 2

Braida Agustin (404621) - Ernst Pedro (400624) - Polizze Tomas (400793) - Valentino Rao (402308)

Técnicas Digitales 2 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional

I. EJERCICIO NRO 2: CONVERSIÓN ANALÓGICA-DIGITAL (ADC)

A. Introducción

Este ejercicio tiene como objetivo comprender el proceso de conversión analógica-digital (ADC) en el microcontrolador STM32F103C8T6. Se implementa la lectura de dos canales: el sensor de temperatura interno del chip (Canal 16) y un canal externo conectado a un potenciómetro (Canal 4 en PA4).

La interfaz de usuario consiste en una barra de 5 LEDs que representan el valor convertido y un botón que alterna entre ambas fuentes de señal. Además, se mantiene el parpadeo del LED integrado (PC13) mediante SysTick, demostrando la ejecución de tareas concurrentes.

B. Funcionamiento Interno del ADC

El microcontrolador STM32F103 utiliza un conversor de aproximaciones sucesivas (SAR - *Successive Approximation Register*). Este proceso transforma una señal analógica de voltaje en un valor digital de 12 bits mediante los siguientes pasos internos:

- 1) **Muestreo y Retención (Sample and Hold):** Antes de iniciar la conversión, un interruptor interno conecta el pin de entrada a un capacitor de muestreo durante un tiempo determinado (configurado mediante los bits SMP). Este capacitor debe cargarse al nivel de voltaje de la entrada. Una vez transcurrido este tiempo, el interruptor se abre, "congelando" el voltaje para el proceso de conversión.
- 2) **Aproximaciones Sucesivas (SAR):** El núcleo del ADC utiliza un registro de aproximación sucesiva, un comparador y un DAC interno. El proceso es similar a una búsqueda binaria:
 - Se activa el bit más significativo (MSB). El DAC genera un voltaje igual a $V_{REF}/2$.
 - El comparador determina si el voltaje de entrada es mayor o menor que este valor.
 - Si es mayor, el bit se mantiene en 1; si es menor, se pone en 0.
 - Este proceso se repite para los 12 bits, desde el MSB hasta el LSB (bit menos significativo).

- 3) **Tiempo Total de Conversión:** El tiempo necesario para obtener un resultado es la suma del tiempo de muestreo (T_{SMPL}) y el tiempo de conversión (T_{CONV}):

$$T_{total} = T_{SMPL} + T_{CONV}$$

Para el STM32F103, el tiempo de conversión es fijo en 12.5 ciclos de reloj del ADC, mientras que el tiempo de muestreo es programable (en este ejercicio se usó el máximo de 239.5 ciclos para mayor precisión).

C. Implementación de la Aplicación (main.c)

La configuración del periférico ADC1 se realiza mediante acceso directo a registros (*Bare Metal*). A continuación se detallan los fragmentos más relevantes del código.

```

1 // 5. Configurar ADC1
2 ADC_CR2 |= (1 << 0); // ADON:
  Encender el ADC
3 ADC_CR2 |= (1 << 23); // TSVREFE:
  Despertar el sensor de Temperatura
  Interno
4
5 // Configurar tiempo de muestreo (El
  sensor de temp requiere un tiempo de
  muestreo largo)
6 ADC_SMPR1 |= (0x7 << 18); // SMP16 =
  111 (239.5 ciclos para temperatura)
7 ADC_SMPR2 |= (0x7 << 12); // SMP4 = 111
  (239.5 ciclos para el potenciómetro
  en PA4)
8
9 // Configurar el Trigger por Software (
  SWSTART)
10 ADC_CR2 |= (0x7 << 17); // EXTSEL = 111
11 ADC_CR2 |= (1 << 20); // EXTTRIG = 1
  (Necesario para arranque por software)

```

Listing 1. Configuración del ADC1 y Sensores en main.c

El bucle principal gestiona la selección del canal, el inicio de la conversión y la actualización de los LEDs:

```

1 while (1)
2 {
3     // Seleccionar el canal segun la
    interrupcion (modo_adc)
4     if (modo_adc == 0) {
5         ADC_SQR3 = 16; // Sensor de
        Temperatura Interno
6     } else {

```

```

7      ADC_SQR3 = 4; // Potenciometro
      en PA4
      }
9
10     ADC_CR2 |= (1 << 22); // Iniciar
      conversion (SWSTART)
11
12     while (!(ADC_SR & (1 << 1))); //
      Esperar EOC (End of Conversion)
13
14     uint16_t adc_val = ADC_DR & 0xFFFF;
      // Leer dato (0 a 4095)
15
16     // Actualizar barra de LEDs
      GPIOA_ODR &= ~(1 << 5) | (1 << 6)
17     | (1 << 7));
      GPIOB_ODR &= ~(1 << 10) | (1 <<
18     11));
19
20     if(adc_val > 682) GPIOA_ODR |= (1
      << 5);
21     if(adc_val > 1365) GPIOA_ODR |= (1
      << 6);
22     if(adc_val > 2048) GPIOA_ODR |= (1
      << 7);
23     if(adc_val > 2730) GPIOB_ODR |= (1
      << 10);
24     if(adc_val > 3412) GPIOB_ODR |= (1
      << 11);
25
26     delay_dummy(200000);
27 }

```

Listing 2. Bucle principal y logica de visualizacion

```

24 .thumb_set EXTIO_IRQHandler,
      Default_Handler
25 .weak EXTI1_IRQHandler
26 .thumb_set EXTI1_IRQHandler,
      Default_Handler
27
28 .section .isr_vector,"a",%progbits
29 .word _esstack
30 .word _reset
31 .word NMI_Handler
32 .word HardFault_Handler
33 // ... (omitiendo vectores reservados)
34 .word SysTick_Handler /* 15: El handler
      del SysTick */
35
36 // Interrupciones de Perifericos (IRQs)
37 .word 0
38 .word 0
39 .word 0
40 .word 0
41 .word 0
42 .word 0
43 .word EXTI0_IRQHandler /* 22: IRQ 6 -
      EXTI0 (Boton PA0) */
44 .word EXTI1_IRQHandler /* 23: IRQ 7 -
      EXTI1 (Boton PA1) */
45
46 .section .text.reset
47 .thumb_func
48 .global _reset
49 _reset:
50     b main
51     b .

```

Listing 3. Tabla de vectores y manejadores en crt.s

D. Archivo de Startup (crt.s)

El archivo `crt.s` es fundamental para la inicialización del sistema y la gestión de interrupciones. En él se define la tabla de vectores, que mapea las direcciones de las rutinas de servicio de interrupción (ISRs) a sus respectivos eventos de hardware.

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores
7
8 // Funcion basica para manejar los punteros
9 .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b . /* Bucle infinito */
13
14 // Definir los vectores internos
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak SysTick_Handler
18 .thumb_set SysTick_Handler, Default_Handler
19 .weak HardFault_Handler
20 .thumb_set HardFault_Handler,
      Default_Handler
21
22 // Definir los vectores para los botones (
      EXTI0 y EXTI1)
23 .weak EXTI0_IRQHandler

```

En esta implementación, se han definido explícitamente los vectores para `EXTIO_IRQHandler` y `EXTI1_IRQHandler`. Esto permite que el hardware salte a las funciones correspondientes definidas en el archivo `main.c` cuando se presionan los botones, facilitando el cambio de modo del ADC y la demostración de prioridades.

E. Análisis Técnico

- **Gestión de Canales:** Se utiliza el registro `ADC_SQR3` para seleccionar el canal de conversión de forma dinámica. La variable `modo_adc` se modifica dentro de la rutina de servicio de interrupción de un botón (`EXTIO_IRQHandler`).
- **Sensor de Temperatura Interno:** Para utilizar este sensor, es imperativo habilitar el bit `TSVREFE` en el registro `ADC_CR2`. Además, debido a la impedancia de salida del sensor, se configuró el tiempo de muestreo al máximo (239.5 ciclos) para asegurar una carga adecuada del capacitor de muestreo y retención del ADC.
- **Conversión por Polling:** El programa espera a que la conversión finalice monitoreando el flag `EOC` (End of Conversion) en el registro de estado (`ADC_SR`). Este es el método más simple de sincronización antes de leer el registro de datos `ADC_DR`.
- **Concurrentemente con SysTick:** El LED de la placa (PC13) continúa parpadeando gracias a la interrupción de `SysTick` configurada a 1ms, demostrando que el proceso

de conversión ADC (que es bloqueante en este ejemplo) no interfiere con las tareas críticas gestionadas por interrupciones.

F. Archivos de Soporte

El proyecto utiliza un `linker.ld` estándar para definir las secciones de memoria Flash y RAM, y un `Makefile` que automatiza la compilación cruzada y la grabación mediante OpenOCD. La tabla de vectores en el archivo de *startup* (`crt.s`) incluye las direcciones de los manejadores para SysTick y EXTI, permitiendo la reactividad del sistema ante eventos externos.

II. CONCLUSIÓN

La implementación del ADC en modo *Bare Metal* permitió profundizar en la configuración de periféricos complejos. Se validó la capacidad del microcontrolador para alternar entre fuentes de señal analógicas (internas y externas) y representar la información mediante salidas digitales. La correcta configuración de los tiempos de muestreo resultó crítica para la estabilidad de las lecturas, especialmente en el caso del sensor de temperatura integrado.